

Department of Electronics and Communication Engineering
Amrita School of Engineering, Amritapuri Campus

SMART PARKING SYSTEM USING ESP32

Group 2

Team Members	Roll Number
N Vivekananda	AM.EN.U4ECE22028
virasith	AM.EN.U4ECE22040

Abstract:

This project is a **3-slot Smart Parking system** built with an ESP32. The ESP32 reads three ultrasonic sensors to detect if a slot is free or occupied and drives three LEDs per slot (green = free, yellow = reserved, red = occupied). A small web server runs on the ESP32 so users on the same Wi-Fi can see slot status and reserve a free slot. Reservations last **2 minutes** and automatically cancel if the car does not arrive.

Objective:

The objective of this project is:

- To build a **3-slot smart parking system** using ESP32.
- To detect if a parking slot is **free or occupied** using ultrasonic sensors.
- To show slot status using **LEDs** (green, yellow, red).
- Provide a simple web page to view and reserve slots over local Wi-Fi.
- To automatically cancel the reservation after **2 minutes** if no vehicle arrives.

Introduction

The system uses an ESP32 microcontroller and three HC-SR04 style ultrasonic sensors. Each slot has one sensor and three LEDs. The ESP32 measures distance using the sensors, smooths readings with a median of samples, decides slot state (FREE, RESERVED, OCCUPIED) and serves a web UI so users can reserve free slots from a browser on the same Wi-Fi network.

Literature Survey Summary:

Paper & Source	Summary	Contribution	Intendified Gap
----------------	---------	--------------	-----------------

1. IoT-Enabled Smart Car Parking System with Route Navigation — Deepak Bhatnagar, Vaibhav Soni, Pragati Agrawal 2024 IEEE	traces the evolution of IoT parking systems, starting from ultrasonic and IR sensor networks to advanced fog/edge-assisted systems and cloud-based backends. The integration of Arduino/Raspberry Pi, Google APIs for mapping, and RFID/NFC/WSN for access and tracking is explored. The survey also covers systems utilizing image-based occupancy detection through Pi cameras and machine learning for slot availability prediction	The transition to advanced parking systems incorporates edge and cloud computing for real-time updates via mobile apps. RFID/NFC systems streamline access and tracking. Using ESP32 Wi-Fi with ATmega2560, IR sensors, and LED route guidance improves user experience and efficiency, especially in dense areas. This approach enhances slot monitoring and navigation, making parking more seamless and efficient.	lack of seamless indoor navigation in multi-level parking systems, with challenges in error-free slot assignment and tight cloud synchronization for live guidance. Improvements are needed to reduce search time, minimize congestion, and enhance the overall user experience.
2. IoT-Based Smart Parking Management System Using ESP32 Microcontroller - Joni Welman Simatupang, Aida Mahdalena Lubis, Vincent (RESEARCH GATE 2022)	Earlier smart parking systems evolved from Bluetooth & ultrasonic sensors to GSM/IR-based systems with power and cost issues. Later systems used IoT stacks like ESP8266/Arduino but required complex mobile apps and multiple controllers. This paper proposes a more streamlined approach using the ESP32 with MQTT, ultrasonic sensors, and RGB slot indicators for real-time monitoring,	The proposed IoT-based ESP32 system reduces complexity and power consumption, eliminating the need for mobile apps by supporting web access. This simplifies the hardware architecture, enhancing usability and scalability in smart parking systems.	Previous systems had high power consumption and complexity, especially in mobile apps and hardware redundancy. Lack of scalable connectivity and simplified systems in earlier solutions. Focus on improving architecture to reduce friction and improve system efficiency. .

	reservation, and status updates.		
3. Design of Solar-IoT Assisted Smart Parking System for Vehicles Using Logical Sensors and Remote Monitoring Facility - S. China Venkateswarlu, Neeraj Kumar, Shiwangi Saini (RESEARCH GATE)	This paper examines IoT systems with UAV imaging, CNN-based vision, and fog/cloud pipelines for occupancy classification. It features Raspberry Pi cameras, Firebase, GSM alerts, and RFID for entry and payments. Challenges in accuracy under varying conditions and the role of cloud analytics are also discussed.	This paper focuses on enhancing occupancy detection accuracy using computer vision and machine learning. It explores integrating UAVs, CNN-based vision, and cloud computing for real-time monitoring. The goal is to reduce errors in occupancy detection and improve responsiveness through cloud analytics.	High installation costs and robustness issues in adverse conditions, such as poor lighting or extreme weather. Challenges include large-scale deployment and maintaining accuracy across varied environments. There is a need for improved cloud analytics and computer vision to handle dynamic parking conditions.
4. A Smart, Efficient, and Reliable Parking Surveillance System with Edge Artificial Intelligence on IoT Devices: Ruimin Ke, Yifan Zhuang, Ziyuan Pu, Yin Hai Wang(arxiv 2024)	This study explores IoT-enabled cameras and edge computing for parking slot monitoring and occupancy detection. It focuses on reducing latency and bandwidth by processing data on-site using an enhanced SSD for vehicle detection, with empirical validation achieving over 95% accuracy in a real-world parking garage.	This study integrates edge AI with IoT for parking management, reducing reliance on cloud processing. It offers a high-accuracy vehicle/slot occupancy detection method, achieving >95% accuracy in real-world deployments, and bridges the gap between sensor-based and AI-enabled smart parking systems.	The paper primarily focuses on occupancy detection using edge-AI, without addressing indoor navigation, slot assignment, or route guidance. It emphasizes detection and surveillance rather than occupant guidance, reservation management, or multi-level facility routing.

Problem Statement

People face difficulty finding empty parking slots in crowded areas. There is no simple way to know:

- Which slots are free

- Which slots are occupied
- Whether a slot can be reserved in advance

This leads to:

- Wasting time searching
- Traffic congestion
- Confusion among drivers

A system is needed to **detect slot availability** and allow **easy reservation**.

Project Proposal:

The project proposes a **Smart Parking System** using:

- ESP32 as the main controller
- Ultrasonic sensors to detect vehicle distance
- LEDs to show slot status (green/free, yellow/reserved, red/occupied)
- A local Wi-Fi web page for users to monitor and reserve slots

How it works:

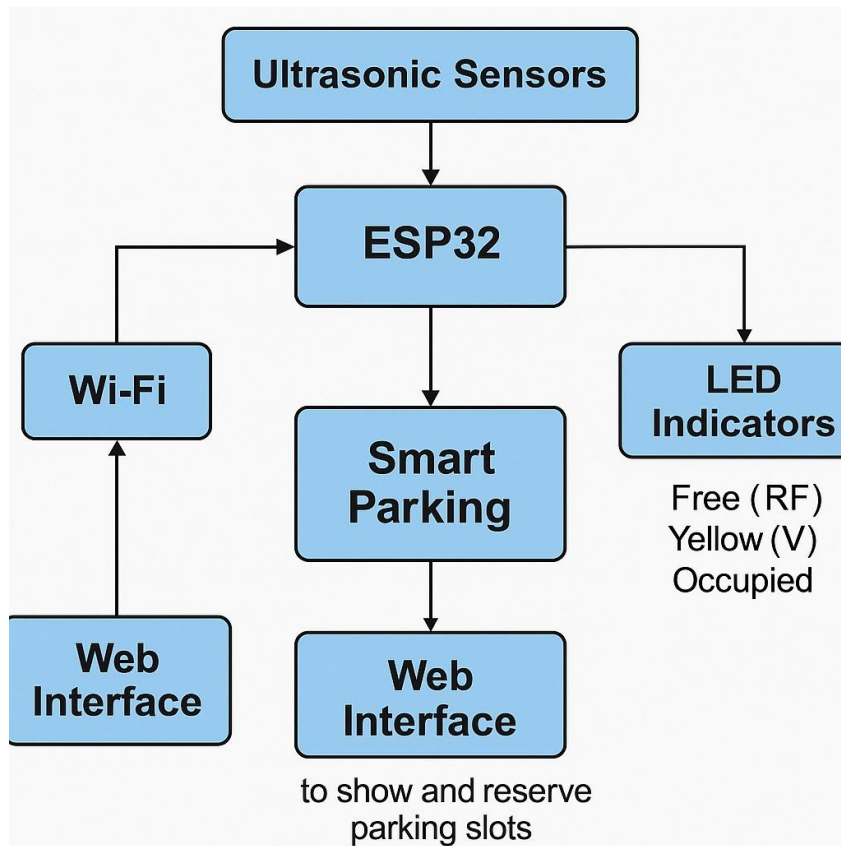
1. ESP32 reads distance from each ultrasonic sensor.
2. If distance $\leq$ threshold $\rightarrow$ slot is **occupied** (red).
3. If distance is large $\rightarrow$ slot is **free** (green).
4. User can open ESP32's web page and click **Reserve**.
5. Reserved slots turn **yellow**.
6. If the car arrives within 2 minutes $\rightarrow$ slot becomes **occupied**.
7. If no car arrives, ESP32 automatically cancels the reservation $\rightarrow$ slot turns green.

The system is low-cost, easy to build, and works completely on local Wi-Fi.

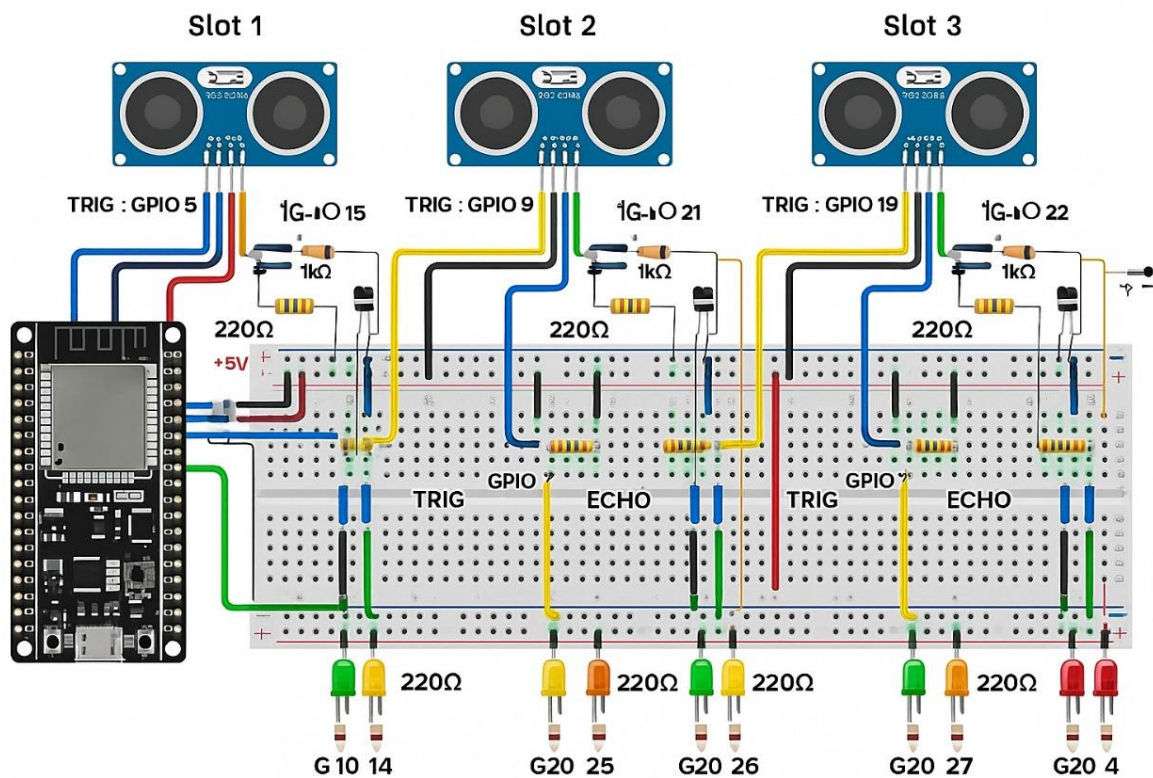
Components Required:

- 1 $\times$ ESP32 development board
- 3 $\times$ Ultrasonic sensors (HC-SR04)
- 9 $\times$ LEDs (Green, Yellow, Red for each slot)
- 9 $\times$ Resistors (220 Ω)
- Breadboard
- Jumper wires

Block Diagram:



Circuit Diagram:



Coding (Arduino – ESP32)

```
/*
  3-slot Smart Parking with web UI (ESP32)
  - WiFi web server serves a simple page to show/reserve slots
  - Slots: FREE (green), RESERVED (yellow), OCCUPIED (red)
  - Reservation timeout: 2 minutes
  - Replace WIFI_SSID and WIFI_PASS below
*/

#include <WiFi.h>
#include <WebServer.h>

// ===== WiFi =====
const char* WIFI_SSID = "Virasith";
const char* WIFI_PASS = "12345678";

WebServer server(80);

// ===== PIN CONFIG (same as your working setup) =====
// SLOT 1
const int trig1 = 5, echo1 = 18;
const int g1 = 12, yel1 = 13, r1 = 14;
// SLOT 2
const int trig2 = 19, echo2 = 21;
const int g2 = 25, yel2 = 26, r2 = 27;
// SLOT 3
const int trig3 = 32, echo3 = 33;
const int g3 = 15, yel3 = 2, r3 = 4;

// ===== SETTINGS =====
const int SAMPLE_COUNT = 7;
const int OCCUPIED_THRESHOLD = 20; // cm
const unsigned long MEASURE_INTERVAL = 200;
const unsigned long RESERVE_TIMEOUT = 120000UL; // 2 minutes

enum SlotState { FREE, RESERVED, OCCUPIED };
SlotState state1 = FREE, state2 = FREE, state3 = FREE;
unsigned long reserveTime1 = 0, reserveTime2 = 0, reserveTime3 = 0;

// ===== DISTANCE FUNCTIONS (same robust approach) =====
long readDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(3);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  long dur = pulseIn(echoPin, HIGH, 30000UL);
```

```

    if (dur == 0) return 200; // treat no echo as far
    long dist = dur * 0.034 / 2;
    if (dist > 200) dist = 200;
    return dist;
}

long getMedian(int trigPin, int echoPin) {
    long samples[SAMPLE_COUNT];
    for (int i = 0; i < SAMPLE_COUNT; i++) {
        samples[i] = readDistance(trigPin, echoPin);
        delay(5);
    }
    // simple sort
    for (int i = 0; i < SAMPLE_COUNT; ++i)
        for (int j = i+1; j < SAMPLE_COUNT; ++j)
            if (samples[j] < samples[i]) {
                long t = samples[i]; samples[i] = samples[j]; samples[j] = t;
            }
    return samples[SAMPLE_COUNT/2];
}

void setLEDsPins(int g, int y, int r, SlotState s) {
    digitalWrite(g, s == FREE);
    digitalWrite(y, s == RESERVED);
    digitalWrite(r, s == OCCUPIED);
}

// ===== WEB PAGE (minimal HTML+JS) =====
const char indexPage[] PROGMEM = R"=====(
<!doctype html>
<html>
<head>
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <title>Smart Parking (ESP32)</title>
    <style>
        body { font-family: Arial, sans-serif; text-align:center; padding:16px; }
        .slot { display:inline-block; width:150px; margin:12px; padding:12px;
border-radius:8px; box-shadow:0 2px 6px rgba(0,0,0,0.15); }
        .dot { width:40px; height:40px; border-radius:50%; margin:8px auto; }
        button { padding:8px 12px; font-size:14px; }
    </style>
</head>
<body>
    <h2>Smart Parking – 3 Slots</h2>
    <div id="slots"></div>
    <p><small>Open from devices on same Wi-Fi</small></p>

<script>

```

```

async function fetchStatus(){
  const r = await fetch('/status');
  const j = await r.json();
  const container = document.getElementById('slots');
  container.innerHTML = '';
  j.slots.forEach((s,i) => {
    const div = document.createElement('div'); div.className='slot';
    const title = document.createElement('div'); title.innerText = 'Slot ' +
(i+1);
    const dot = document.createElement('div'); dot.className='dot';
    if(s.state=='FREE') dot.style.background='green';
    else if(s.state=='RESERVED') dot.style.background='gold';
    else if(s.state=='OCCUPIED') dot.style.background='red';
    const info = document.createElement('div'); info.innerText = s.state +
(s.dist?s.dist+' cm':'');
    const btn = document.createElement('button');
    if(s.state=='FREE') { btn.innerText='Reserve'; btn.onclick =
()=>reserve(i+1); }
    else { btn.innerText='--'; btn.disabled = true; }
    div.appendChild(title); div.appendChild(dot); div.appendChild(info);
div.appendChild(btn);
    container.appendChild(div);
  });
}

async function reserve(slot){
  await fetch('/reserve?slot=' + slot);
  setTimeout(fetchStatus, 300);
}

setInterval(fetchStatus, 1500);
fetchStatus();
</script>
</body>
</html>
)=====";

// ===== WEB HANDLERS =====
void handleRoot() {
  server.send_P(200, "text/html", indexPage);
}

void handleStatus() {
  // return JSON with states & distances
  String json = "{ \"slots\": [";
  // s1
  long d1 = getMedian(trig1, echo1);

```



```

    json += String("{\"state\":" +
(state1==FREE?"FREE":state1==RESERVED?"RESERVED":"OCCUPIED") + "\", \"dist\":"
+ d1 + "},";
    long d2 = getMedian(trig2, echo2);
    json += String("{\"state\":" +
(state2==FREE?"FREE":state2==RESERVED?"RESERVED":"OCCUPIED") + "\", \"dist\":"
+ d2 + "},";
    long d3 = getMedian(trig3, echo3);
    json += String("{\"state\":" +
(state3==FREE?"FREE":state3==RESERVED?"RESERVED":"OCCUPIED") + "\", \"dist\":"
+ d3 + "}";
    json += "]}";
    server.send(200, "application/json", json);
}

void handleReserve() {
    if (!server.hasArg("slot")) { server.send(400, "text/plain", "missing
slot"); return; }
    int s = server.arg("slot").toInt();
    if (s < 1 || s > 3) { server.send(400, "text/plain", "invalid slot");
return; }

    if (s==1) {
        if (state1 == FREE) {
            state1 = RESERVED; reserveTime1 = millis();
            setLEDPins(g1,yel1,r1,RESERVED);
            server.send(200, "text/plain", "reserved 1");
            return;
        } else server.send(409, "text/plain", "not free");
    }
    if (s==2) {
        if (state2 == FREE) { state2 = RESERVED; reserveTime2 = millis();
setLEDPins(g2,yel2,r2,RESERVED); server.send(200,"text/plain","reserved 2");
return; }
        else server.send(409,"text/plain","not free");
    }
    if (s==3) {
        if (state3 == FREE) { state3 = RESERVED; reserveTime3 = millis();
setLEDPins(g3,yel3,r3,RESERVED); server.send(200,"text/plain","reserved 3");
return; }
        else server.send(409,"text/plain","not free");
    }
}

void setupWeb() {
    server.on("/", handleRoot);
    server.on("/status", handleStatus);
    server.on("/reserve", handleReserve);
}

```

```

    server.begin();
}

// ===== SETUP =====
void setup() {
    Serial.begin(115200);

    // init pins
    pinMode(trig1, OUTPUT); pinMode(echo1, INPUT);
    pinMode(trig2, OUTPUT); pinMode(echo2, INPUT);
    pinMode(trig3, OUTPUT); pinMode(echo3, INPUT);

    pinMode(g1, OUTPUT); pinMode(yel1, OUTPUT); pinMode(r1, OUTPUT);
    pinMode(g2, OUTPUT); pinMode(yel2, OUTPUT); pinMode(r2, OUTPUT);
    pinMode(g3, OUTPUT); pinMode(yel3, OUTPUT); pinMode(r3, OUTPUT);

    setLEDPins(g1,yel1,r1,FREE);
    setLEDPins(g2,yel2,r2,FREE);
    setLEDPins(g3,yel3,r3,FREE);

    // connect WiFi
    WiFi.begin(WIFI_SSID, WIFI_PASS);
    Serial.print("Connecting to WiFi");
    int tries=0;
    while (WiFi.status() != WL_CONNECTED && tries < 40) {
        delay(250);
        Serial.print(".");
        tries++;
    }
    if (WiFi.status() == WL_CONNECTED) {
        Serial.println();
        Serial.print("Connected! IP: ");
        Serial.println(WiFi.localIP());
    } else {
        Serial.println();
        Serial.println("WiFi failed - AP mode not implemented; check creds");
    }
    setupWeb();
    Serial.println("Web server started. Open browser to the IP shown above.");
}

// helper: handle reservation timeout
void checkReservationTimeout() {
    if (state1 == RESERVED && millis() - reserveTime1 > RESERVE_TIMEOUT) {
        state1 = FREE; setLEDPins(g1,yel1,r1,FREE); Serial.println("Slot1 reserve
expired"); }
}

```

```

    if (state2 == RESERVED && millis() - reserveTime2 > RESERVE_TIMEOUT) {
state2 = FREE; setLEDPins(g2,yel2,r2,FREE); Serial.println("Slot2 reserve
expired"); }
    if (state3 == RESERVED && millis() - reserveTime3 > RESERVE_TIMEOUT) {
state3 = FREE; setLEDPins(g3,yel3,r3,FREE); Serial.println("Slot3 reserve
expired"); }
}

void loop() {
    server.handleClient();           // handle web requests

    // read sensors and update states (but keep reservation priority)
    long d1 = getMedian(trig1, echo1);
    long d2 = getMedian(trig2, echo2);
    long d3 = getMedian(trig3, echo3);

    // only change states if not RESERVED
    if (state1 != RESERVED) {
        state1 = (d1 <= OCCUPIED_THRESHOLD) ? OCCUPIED : FREE;
        setLEDPins(g1,yel1,r1,state1);
    } else {
        // check if car arrived while reserved
        if (d1 <= OCCUPIED_THRESHOLD) { state1 = OCCUPIED;
setLEDPins(g1,yel1,r1,OCCUPIED); Serial.println("Slot1 reserved -> car
arrived"); }
    }
    if (state2 != RESERVED) {
        state2 = (d2 <= OCCUPIED_THRESHOLD) ? OCCUPIED : FREE;
        setLEDPins(g2,yel2,r2,state2);
    } else {
        if (d2 <= OCCUPIED_THRESHOLD) { state2 = OCCUPIED;
setLEDPins(g2,yel2,r2,OCCUPIED); Serial.println("Slot2 reserved -> car
arrived"); }
    }
    if (state3 != RESERVED) {
        state3 = (d3 <= OCCUPIED_THRESHOLD) ? OCCUPIED : FREE;
        setLEDPins(g3,yel3,r3,state3);
    } else {
        if (d3 <= OCCUPIED_THRESHOLD) { state3 = OCCUPIED;
setLEDPins(g3,yel3,r3,OCCUPIED); Serial.println("Slot3 reserved -> car
arrived"); }
    }

    checkReservationTimeout();

    delay(MEASURE_INTERVAL);
}

```

Result

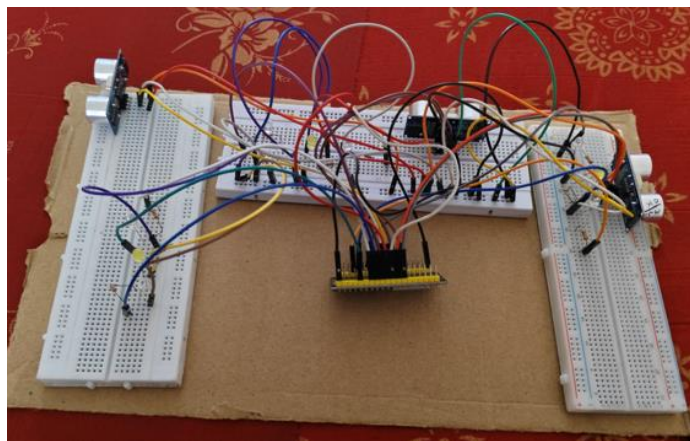
- The system successfully detects **three parking slots** using ultrasonic sensors.
- The LED for each slot correctly shows the status:

Green → Free

Yellow → Reserved

Red → Occupied

- A fully working **web interface** is hosted by the ESP32.
- Users can open the web page from their phone and **reserve a free slot**.
- The reserved slot turns **yellow**, and if the car arrives, the slot becomes **red**.
- If the car doesn't arrive in **2 minutes**, the reservation **automatically expires**.
- All readings and states update in **real time** without refreshing the webpage.



Conclusion

This project successfully demonstrates a simple and effective **IoT-based Smart Parking System**.

Using only an ESP32, ultrasonic sensors, LEDs, and a small web interface, the system can detect slot availability and allow users to reserve parking slots conveniently.

The reservation timeout ensures fair usage, and real-time monitoring makes the system user-friendly and efficient.

This project can be expanded further with features like mobile apps, cloud dashboards, and automated gates.

Overall, the system reduces waiting time, avoids confusion, and improves the parking experience.